STUDENT INFORMATION DICTIONARY

1. Introduction

The Student Information Dictionary is a Python-based project designed to manage student records using dictionaries. It allows users to search for students, add new records, update existing information, and display all available records.

The project demonstrates how Python dictionaries can store structured information using key-value pairs.

2. Objective

The objectives of this project are:

- To understand dictionaries in Python.
- To represent student records using key-value pairs.
- To search for student information using unique IDs.
- To add new student records.
- To update existing student details.
- To practise functions, loops, conditional statements, and validation.

3. Problem Statement

Managing student records requires an organized way to store and retrieve information. Searching and updating records manually can be inefficient.

This project uses a Python dictionary to organize student information and provides a menu-driven program for basic record management.

4. Tools and Technologies Used

Python 3: Used to develop the student record management program.

Visual Studio Code: Used to write, edit, and execute the Python program.

GitHub: Used to store the source code and project documentation.

5. Data Structure

The project uses a dictionary in which each student ID serves as a unique key. The corresponding value is another dictionary containing the student's details.

The stored fields include:

- Student ID
- Name
- Age
- Course

- Marks

The program uses manually created fictional student records for demonstration.

6. Methodology

The project follows these steps:

1. Create a dictionary containing sample student records.
2. Display a menu of available operations.
3. Accept the user's selected operation.
4. Search for a student using the student ID.
5. Add a new record after validating its details.
6. Update the details of an existing student.
7. Display all stored student records.
8. Handle invalid menu choices and invalid data.
9. Allow the user to exit the program.

7. Features

Search Student

Searches the dictionary using a student ID and displays the corresponding information when found.

Add Student

Creates a new record containing the student's name, age, course, and marks. Duplicate student IDs are rejected.

Update Student

Allows existing student information to be modified while retaining unchanged values.

Display All Students

Displays all student records currently stored in the dictionary.

Input Validation

Checks required fields, positive age values, and marks within the range of 0 to 100.

8. Sample Records

The initial sample dataset contains the following fictional records:

Student ID	Name	Age	Course	Marks

S101	Arun	20	Python	85
S102	Priya	21	Data Science	91
S103	Rahul	19	Artificial Intelligence	78

9. Results and Output

The program displays a menu that allows the user to search, add, update, and display student records.

When searching for an existing ID, the student's details are displayed. When adding a valid new student, a success message appears. Updates are reflected in the current session.

Screenshot Evidence: Insert a screenshot of the actual program output here.

10. Testing and Validation

The program should be tested using different scenarios, including:

- Searching for an existing student.
- Searching for an unknown student ID.
- Adding a new student.
- Attempting to add a duplicate ID.
- Updating an existing student.
- Entering invalid age or marks.
- Displaying all records.
- Entering an invalid menu option.
- Exiting the program.

Actual test results should be recorded after executing the tests.

11. Concepts Learned

This project demonstrates:

- Python dictionaries
- Key-value pairs
- Nested dictionaries
- Functions
- Loops and conditional statements
- Input validation
- Exception handling
- Basic record management

12. Limitations and Future Improvements

The current program stores records in memory, so changes are lost when the program exits.

Future improvements could include:

- Saving records to JSON or CSV files.
- Adding a delete-student operation.
- Implementing a graphical user interface.
- Adding sorting and searching by name.
- Connecting the program to a database.

13. Conclusion

The Student Information Dictionary project demonstrates how Python dictionaries can be used to manage structured student information. It provides practical experience with searching, adding, updating, and displaying records.

The project strengthens foundational Python skills and provides a starting point for more advanced data management applications.

14. Project Links

GitHub Repository: https://github.com/venky1318/student-information-dictionary